

Software Development Methodologies

Waterfall Model

Software Engineering Review

Software Engineering is the systematic approach to developing software.

It applies engineering principles throughout the software lifecycle.

The major activities include:

- Requirements Gathering
- System Design
- Implementation (Coding)
- Testing
- Deployment
- Maintenance

Question

How should these activities be organized to develop software efficiently?

A **Software Development Methodology** defines the process used to develop software.

It specifies:

- How work is organized
- Roles and responsibilities
- Documentation
- Communication
- Development activities

Goal

Deliver high-quality software on time and within budget.

What is a Methodology?

A methodology is a formal set of practices followed during software development.

It provides:

- Standard development process
- Rules and guidelines
- Project organization
- Better communication
- Improved software quality

Examples

- Waterfall
- Agile
- Spiral
- Incremental
- Extreme Programming (XP)

The Waterfall Model

The Waterfall Model is the earliest and one of the simplest software development methodologies.

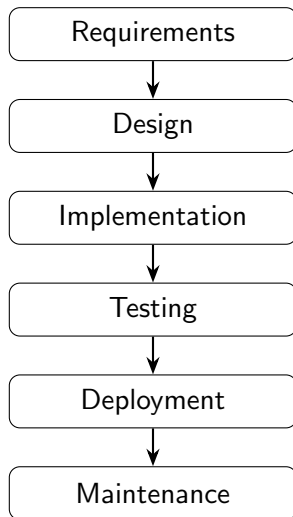
Characteristics:

- Sequential development
- One phase is completed before the next begins
- Heavy documentation
- Suitable for stable requirements

Idea

Each phase flows downward like a waterfall.

Waterfall Development Process



The output of one phase becomes the input to the next phase.

Formal Waterfall Process

Real software projects involve much more than coding.

Each phase produces important documents.

Typical activities include:

- System Requirements
- Software Requirements Specification (SRS)
- Preliminary Design
- Design Review
- Final Design
- Coding
- Testing
- Deployment
- Operations and Maintenance

Note

Review meetings are conducted before moving to the next phase to detect errors early. Although Waterfall is sequential, limited feedback between adjacent phases may occur to correct errors.

Advantages of Waterfall

Advantages include:

- Easy to understand
- Well-defined phases
- Simple project management
- Clear documentation
- Easy to measure progress
- Suitable for small projects with fixed requirements

Disadvantages of Waterfall

Disadvantages include:

- Difficult to accommodate changing requirements
- Customer involvement is limited
- Testing occurs late
- Errors may be expensive to fix
- Working software is delivered only at the end

Major Limitation

If customer requirements change during development, returning to earlier phases becomes costly.

When Should We Use Waterfall?

Waterfall is suitable when:

- Requirements are clear and stable
- Technology is well understood
- Project scope is fixed
- Strong documentation is required
- Customer requirements are unlikely to change

Examples:

- Government projects
- Banking systems
- Defense software
- Regulatory applications

- A methodology defines how software is developed.
- Waterfall is a sequential software development model.
- Each phase must be completed before the next begins.
- Documentation and reviews are important.
- Waterfall works best when requirements are stable.

Limitations of the Waterfall Model

Although Waterfall is simple, it has several limitations.

Common problems include:

- Requirements may change during development.
- Customers see the software only at the end.
- Errors discovered late are expensive to fix.
- Testing begins after coding is completed.
- Long development cycle before delivery.

Result

These limitations led to the development of more flexible software development methodologies such as Agile.

Agile is a modern software development methodology that emphasizes flexibility and customer collaboration.

Instead of developing the entire software at once, Agile delivers software in small increments.

Characteristics:

- Incremental development
- Continuous customer feedback
- Frequent software releases
- Adaptation to changing requirements

Why Agile?

Agile was introduced to solve many problems of the Waterfall model.

Agile helps by:

- Accepting changing requirements
- Delivering software quickly
- Improving communication
- Encouraging customer participation
- Reducing project risk

Main Goal

Deliver working software early and improve it continuously.

The Agile Manifesto

The Agile Manifesto was introduced in 2001.

It values:

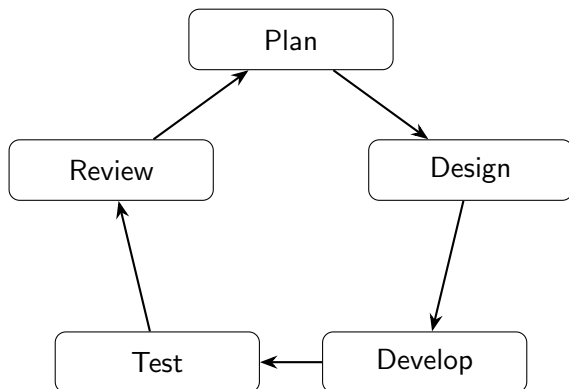
- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

The Agile Spirit

Agile development encourages:

- Customer satisfaction
- Small and frequent releases
- Team collaboration
- Simplicity
- Continuous improvement
- Flexibility
- Rapid feedback
- High-quality software

Agile Development Cycle



Each iteration produces a working version of the software.

Several methodologies follow Agile principles.

Common Agile methodologies include:

- Scrum
- Extreme Programming (XP)
- Feature Driven Development (FDD)
- Crystal
- Dynamic System Development Method (DSDM)
- Adaptive Software Development (ASD)

In this lecture, we will focus on **Extreme Programming (XP)**.

Waterfall vs Agile

Waterfall	Agile
Sequential development	Iterative development
Requirements fixed	Requirements may change
Testing after coding	Continuous testing
Customer involved mainly at the beginning	Customer involved throughout
One final delivery	Frequent releases
Heavy documentation	Lightweight documentation

Waterfall vs Agile

Waterfall

- Best for stable requirements
- Easier planning
- Less customer interaction
- Difficult to make changes

Agile

- Best for changing requirements
- Frequent customer feedback
- Faster delivery
- Easier to adapt to changes

- Agile is a flexible software development methodology.
- It delivers software in small increments.
- Customer collaboration is an important part of Agile.
- Agile welcomes changing requirements.
- Extreme Programming (XP) is one of the most popular Agile methodologies.

Extreme Programming (XP)

Extreme Programming (XP) is one of the most popular Agile software development methodologies.

It was introduced by **Kent Beck** in 1999.

XP focuses on:

- Customer satisfaction
- Frequent software releases
- High-quality code
- Team collaboration
- Continuous improvement

Goal

Develop high-quality software while adapting quickly to changing requirements.

Core Values of XP

Extreme Programming is based on five core values.

- Communication
- Simplicity
- Feedback
- Courage
- Respect

These values guide all XP practices and encourage teamwork and continuous improvement.

The 12 Practices of XP

XP consists of twelve important practices.

- 1 Planning Game
- 2 Small Releases
- 3 Metaphor
- 4 Simple Design
- 5 Test-Driven Development (TDD)
- 6 Refactoring
- 7 Pair Programming
- 8 Collective Code Ownership
- 9 Continuous Integration
- 10 40-Hour Work Week
- 11 On-site Customer
- 12 Coding Standards

Planning Game

- Requirements are written as User Stories.
- Customers prioritize the features.
- Developers estimate effort and time.
- Team decides what will be developed next.

Small Releases

- Deliver software frequently.
- Each release provides business value.
- Customer feedback is obtained early.

Metaphor

- Uses a common story or analogy.
- Helps everyone understand the system.
- Improves communication among team members.

Simple Design

- Design only what is needed now.
- Avoid unnecessary complexity.
- Improve the design later through refactoring.

Test-Driven Development (TDD)

In XP, testing begins before coding.

Development process:

- 1 Write a test.
- 2 Run the test (it fails).
- 3 Write the code.
- 4 Run the test again.
- 5 Refactor if necessary.

Benefits:

- Better software quality
- Early bug detection
- Easier maintenance

Refactoring means improving existing code without changing its behavior.
Common reasons for refactoring:

- Remove duplicate code
- Improve readability
- Simplify complex logic
- Reduce code complexity
- Improve maintainability

Remember

Refactoring improves code quality without changing program output.

Pair Programming

Two programmers work together at one computer.

Roles:

- Driver writes the code.
- Navigator reviews the code continuously.

Benefits:

- Better code quality
- Fewer bugs
- Knowledge sharing
- Faster problem solving

Collective Code Ownership

- Everyone owns the code.
- Any developer can modify any module.
- Reduces dependency on individuals.

Continuous Integration

- Code is integrated frequently.
- Automated tests run after every integration.
- Problems are detected early.

40-Hour Work Week

- Avoid excessive overtime.
- Maintain productivity and quality.

On-site Customer

- Customer works closely with the team.
- Requirements are clarified immediately.

Coding Standards

- Follow common coding conventions.
- Produce readable and maintainable code.

Benefits of Extreme Programming

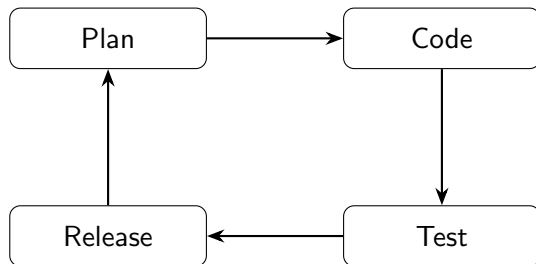
Advantages of XP include:

- High-quality software
- Continuous customer feedback
- Faster delivery
- Reduced development risk
- Better teamwork
- Easier maintenance
- Improved customer satisfaction

- XP is an Agile software development methodology.
- It emphasizes communication, simplicity, and feedback.
- XP follows twelve best practices.
- Continuous testing and integration improve software quality.
- XP is suitable for projects with changing requirements.

How XP Works

The XP development cycle is short and repetitive.



Each iteration produces working software and incorporates customer feedback for the next iteration.

Daily Stand-up Meeting

Many Agile teams begin each day with a short stand-up meeting. Each team member answers three questions:

- 1 What did I complete yesterday?
- 2 What will I work on today?
- 3 Am I facing any obstacles?

Benefits:

- Better communication
- Early problem detection
- Improved coordination

Face-to-Face Communication

XP encourages direct communication among team members.

Advantages:

- Faster decision making
- Fewer misunderstandings
- Better collaboration
- Immediate feedback

Remember

The richest communication occurs through face-to-face discussion, especially while solving complex problems.

How XP Solves Software Engineering Problems

Problem	XP Solution
Changing requirements	Small releases and customer feedback
High defect rate	Test-Driven Development
Late bug detection	Continuous Integration
Poor communication	Pair Programming
Knowledge concentrated in one developer	Collective Code Ownership
Slow delivery	Frequent iterations

When Should XP Be Used?

XP is most suitable when:

- Requirements change frequently
- Customer feedback is available
- Project size is small or medium
- Developers work closely together
- Fast delivery is important

Examples:

- Startup projects
- Web applications
- Mobile applications
- Prototype development

XP is not suitable for every project.

Limitations include:

- Difficult to manage large teams
- Requires continuous customer involvement
- Pair programming may increase cost
- Frequent meetings require discipline
- Requires experienced developers

Waterfall vs XP

Waterfall	XP
Sequential process	Iterative process
Requirements fixed	Requirements change frequently
Testing after coding	Testing before and during coding
Documentation focused	Working software focused
One final release	Frequent small releases
Limited customer interaction	Continuous customer collaboration

Overall Summary

- Software development follows different methodologies.
- Waterfall is a sequential and document-driven model.
- Agile provides flexibility through iterative development.
- Extreme Programming (XP) is an Agile methodology.
- XP emphasizes testing, teamwork, communication, and continuous improvement.

Remember

- Choose the methodology based on project requirements.
- Waterfall works well for stable requirements.
- Agile and XP work well for changing requirements.
- Customer feedback and teamwork are essential for Agile success.
- Good software is achieved through proper process, not just good coding.

Thank You!

Questions or Discussion